

校园中转分发与跑腿服务管理系统——从零开始10人分工方案

一、重新分工背景

本项目目前处于 **尚未开始开发** 状态，因此原方案中“总集成负责人已经完成项目框架、数据库连接、登录权限、基础资料模块”等前提不再成立。

新的分工应按照 **从 0 到可运行、可联调、可答辩演示** 的思路重新设计。

本项目采用：

ASP.NET Core MVC + C# + Oracle 19c + Bootstrap

数据库以课程设计给定的 **24 张表中文枚举版** 为准，所有业务状态值统一使用中文枚举。

二、重新分工原则

本次不再采用“前端等后端、后端等数据库”的线性分工，而采用：

1名总集成与架构负责人
+ 1名数据库与数据负责人
+ 7名业务模块负责人
+ 1名测试文档与答辩负责人

核心原则如下：

1. 项目从零开始，先完成基础框架、数据库脚本、登录权限和公共页面；
2. 每个业务模块负责人采用“全栈负责制”，负责自己模块的 Controller、Service、Repository、ViewModel、View 和基本测试；
3. 公共框架、数据库连接、Layout、菜单、权限判断由专人统一维护，其他人不得重复搭建项目；
4. 数据库表结构、中文枚举、外键关系由数据库负责人统一确认，避免每个人写一套字段名；
5. 每个模块先完成最小可用功能，再补充美化、异常提示、统计报表；
6. 最终目标不是功能越多越好，而是跑通一条完整演示闭环；
7. 所有人都要能讲清自己负责的表、页面、业务逻辑和数据库变化；
8. 所有代码提交前必须能 `dotnet build` 通过；
9. 所有页面状态、按钮、提示信息尽量使用中文；
10. 不允许使用英文枚举值替代数据库中的中文状态值。

三、项目最小演示闭环

优先保证以下流程可以跑通：

管理员初始化基础数据
↓
普通用户注册 / 登录



如果时间紧，优先完成“发布任务 -> 支付 -> 接单 -> 状态流转 -> 签收 -> 评价”的主流程。

四、10人分工总表

组 员	角色定 位	主要负责内容	核心产出	状 态
组 员1	总集成 与架构 负责人	ASP.NET Core MVC 项目搭 建、Oracle 连接、公共 Layout、菜单、权限过滤、代 码合并、最终联调	可运行项目骨架、公共工具类、 最终整合版本	待 开 始
组 员2	数据库 与基础 数据负 责人	24 张表建表脚本、中文枚举检 查、测试数据、基础资料初始 化、数据库说明	建表 SQL、初始化 SQL、测试数 据 SQL、数据库字典	待 开 始
组 员3	账户与 权限模 块负责 人	用户注册登录、角色权限、用 户信息、地址管理、账号状态	users、user_addresses 相关功 能	待 开 始
组 员4	基础资 料与跑 腿员模 块负责 人	节点管理、服务类型、服务节 点规则、跑腿员申请与审核	nodes、service_types、 service_node_rules、runners 相关功能	待 开 始
组 员5	任务发 布模块 负责人	任务主单、三类任务明细、我 的任务、任务详情、任务取消	tasks 及三类 detail 表相关功能	待 开 始

组 员	角色定 位	主要负责内容	核心产出	状 态
组 员6	接单派 单与状 态流转 负责人	任务大厅、抢单、管理员派 单、重派、状态更新、状态日 志	assign_records、 task_status_logs 相关功能	待 开 始
组 员7	支付退 款模块 负责人	模拟支付、支付记录、退款申 请、退款审核、支付状态联动	payments、refunds 相关功能	待 开 始
组 员8	评价投 诉与信 誉模块 负责人	用户评价、用户投诉、管理员 处理投诉、跑腿员信誉分变化	reviews、complaints、runners 相关功能	待 开 始
组 员9	结算审 计报表 模块负 责人	跑腿员结算、结算明细、审计 日志、统计报表	settlements、audit_logs、 reports 相关功能	待 开 始
组 员 10	测试文 档与答 辩负责 人	功能测试、Bug 记录、截图整 理、系统文档、PPT、答辩演示 脚本	测试报告、演示截图、答辩材料	待 开 始

五、阶段安排

第1阶段：项目启动与公共基础

建议周期：第 1 天到第 2 天。

任务	负责人	验收标准
创建 ASP.NET Core MVC 项目	组员1	项目可以启动并显示首页
配置 Oracle 连接	组员1、组 员2	能连接 Oracle 并执行简单查询
建立 24 张表	组员2	SQL Developer 中可以看到全部表
建立基础测试数据	组员2	至少有管理员、普通用户、跑腿员、节点、服务类型 数据
设计公共 Layout	组员1	有顶部栏、侧边栏、角色菜单
建立分层目录	组员1	Controllers、Services、Repositories、 ViewModels 目录清晰
约定中文枚举常量	组员1、组 员2	形成统一枚举说明，不允许各写各的

第2阶段：基础模块开发

建议周期：第 3 天到第 5 天。

任务	负责人	依赖
注册登录与 Session	组员3	组员1 项目骨架
角色权限控制	组员3、组员1	登录模块
地址管理	组员3	users、user_addresses 表
节点管理	组员4	nodes 表
服务类型管理	组员4	service_types 表
服务节点规则管理	组员4	nodes、service_types 表
跑腿员申请与审核	组员4	users、runners 表

第3阶段：核心业务闭环开发

建议周期：第 6 天到第 9 天。

任务	负责人	依赖
发布外卖分发任务	组员5	地址、服务类型、节点规则
发布快递代取任务	组员5	地址、服务类型、节点规则
发布私人任务	组员5	地址、服务类型、节点规则
我的任务与任务详情	组员5	任务主单和明细
模拟支付	组员7	任务发布
任务大厅	组员6	已支付或待接单任务
跑腿员抢单	组员6	跑腿员审核通过
任务状态流转	组员6	接单记录
用户确认签收	组员6	状态流转
用户评价	组员8	已完成任务

第4阶段：扩展功能与管理员处理

建议周期：第 10 天到第 12 天。

任务	负责人	依赖
退款申请	组员7	支付记录

任务	负责人	依赖
退款审核	组员7	管理员权限
用户投诉	组员8	接单记录
投诉处理	组员8	管理员权限
信誉分扣减	组员8	投诉成立
跑腿员结算	组员9	已完成任务和支付记录
审计日志	组员9	支付、退款、状态日志
统计报表	组员9	任务、支付、投诉、结算数据

第5阶段：联调、测试与答辩

建议周期：第 13 天到第 15 天。

任务	负责人	验收标准
合并各模块代码	组员1	主分支可运行
全流程联调	组员1、组员10、全体	演示闭环能完整跑通
数据库结果验证	组员2、组员10	SQL Developer 可查到关键数据变化
Bug 记录与回归测试	组员10	每个 Bug 有状态记录
截图整理	组员10、全体	每个模块至少 1 到 2 张截图
PPT 内容汇总	组员10	每人负责部分完整
答辩演示脚本	组员10、组员1	演示步骤清晰

六、各组员详细分工

组员1：总集成与架构负责人

负责范围

组员1负责项目从零搭建和最终整合，是整个项目能否顺利运行的第一责任人。

主要任务

1. 创建 ASP.NET Core MVC 项目；
2. 配置项目依赖；
3. 配置 Oracle 数据库连接；
4. 编写或整理 `OracleDbHelper`；
5. 建立统一目录结构；
6. 设计公共 Layout；

7. 设计顶部栏、侧边栏、首页；
8. 设计登录后角色菜单显示规则；
9. 维护公共错误提示页面；
10. 维护 Git 分支和代码合并；
11. 处理项目运行错误；
12. 组织最终联调。

建议负责文件

```
Program.cs
appsettings.json
Helpers/OracleDbHelper.cs
Controllers/HomeController.cs
Views/Shared/_Layout.cshtml
Views/Shared/_LoginPartial.cshtml
wwwroot/css/site.css
```

不建议承担

1. 不单独包揽所有业务模块；
2. 不替每个人重写业务代码；
3. 不随意改动数据库表结构；
4. 不在未沟通的情况下修改其他成员模块逻辑。

验收标准

1. 项目可以启动；
2. 首页可以访问；
3. Oracle 数据库连接成功；
4. 公共 Layout 正常显示；
5. 登录后能区分普通用户、跑腿员、管理员；
6. 合并后 `dotnet build` 通过。

答辩讲解内容

1. 系统整体架构；
 2. ASP.NET Core MVC 分层结构；
 3. Oracle 连接方式；
 4. 权限控制方式；
 5. 项目联调与整合过程。
-

组员2：数据库与基础数据负责人

负责范围

组员2负责数据库从零建立、字段说明、中文枚举约束、测试数据和数据库验收。

主要任务

1. 整理 24 张表建表 SQL；
2. 确认主键、外键、唯一约束；
3. 确认所有中文枚举值；
4. 编写基础数据 SQL；
5. 编写测试数据 SQL；
6. 整理数据库表字段说明；
7. 给其他成员提供字段名和枚举说明；
8. 协助排查 Oracle 字段名、类型、约束错误；
9. 每个模块完成后协助做 SQL Developer 验证；
10. 准备数据库设计答辩内容。

重点数据

管理员账号
普通用户账号
跑腿员账号
校内节点
服务类型
服务节点规则
示例地址
示例任务
示例支付记录
示例评价投诉

建议负责文件

Database/create_tables.sql
Database/init_base_data.sql
Database/init_test_data.sql
Docs/database_dictionary.md
Docs/chinese_enum_values.md

验收标准

1. 24 张表可以成功创建；
2. 基础数据可以成功插入；
3. 外键关系正确；
4. 中文枚举值明确；
5. 其他成员能根据文档正确写 SQL；

6. 答辩时能讲清核心表关系。

组员3：账户与权限模块负责人

负责表

```
users
user_addresses
```

负责功能

- 1. 用户注册；
- 2. 用户登录；
- 3. 用户退出；
- 4. Session 保存用户身份；
- 5. 角色权限判断；
- 6. 用户信息查看；
- 7. 用户信息修改；
- 8. 账号禁用判断；
- 9. 地址列表；
- 10. 新增地址；
- 11. 修改地址；
- 12. 删除地址；
- 13. 设置默认地址。

中文枚举要求

字段	可用值
users.user_role	普通用户、跑腿员、管理员
users.account_status	正常、禁用
user_addresses.is_default	是、否

建议负责文件

```
Controllers/AuthController.cs
Controllers/UserController.cs
Controllers/AddressController.cs
Services/UserService.cs
Services/AddressService.cs
Repositories/UserRepository.cs
Repositories/AddressRepository.cs
ViewModels/LoginViewModel.cs
ViewModels/RegisterViewModel.cs
ViewModels/UserViewModel.cs
```

```
ViewModels/AddressViewModel.cs
Views/Auth/*
Views/User/*
Views/Address/*
```

验收标准

- 1. 普通用户可以注册;
- 2. 用户可以登录和退出;
- 3. 禁用账号不能登录;
- 4. 用户可以维护多个地址;
- 5. 同一用户只能有一个默认地址;
- 6. 数据库 `users` 和 `user_addresses` 变化正确。

组员4：基础资料与跑腿员模块负责人

负责表

```
nodes
service_types
service_node_rules
runners
```

负责功能

- 1. 节点列表;
- 2. 新增节点;
- 3. 修改节点;
- 4. 删除或停用节点;
- 5. 服务类型列表;
- 6. 新增服务类型;
- 7. 修改服务类型;
- 8. 服务节点规则绑定;
- 9. 跑腿员申请;
- 10. 跑腿员审核;
- 11. 跑腿员工作状态维护;
- 12. 跑腿员信息查看。

中文枚举要求

字段	可用值
nodes.node_status	正常、关闭
service_types.service_status	启用、禁用

字段	可用值
runners.audit_status	待审核、已通过、已拒绝
runners.work_status	空闲、忙碌、离线

建议负责文件

```
Controllers/NodeController.cs
Controllers/ServiceTypeController.cs
Controllers/ServiceNodeRuleController.cs
Controllers/RunnerController.cs
Services/NodeService.cs
Services/ServiceTypeService.cs
Services/ServiceNodeRuleService.cs
Services/RunnerService.cs
Repositories/NodeRepository.cs
Repositories/ServiceTypeRepository.cs
Repositories/ServiceNodeRuleRepository.cs
Repositories/RunnerRepository.cs
Views/Node/*
Views/ServiceType/*
Views/ServiceNodeRule/*
Views/Runner/*
```

验收标准

- 1. 管理员可以维护节点；
- 2. 管理员可以维护服务类型；
- 3. 管理员可以绑定服务类型和节点；
- 4. 普通用户可以申请成为跑腿员；
- 5. 管理员可以审核跑腿员；
- 6. 审核通过后跑腿员可以参与接单流程。

组员5：任务发布模块负责人

负责表

```
tasks
food_delivery_details
express_pickup_details
private_task_details
service_node_rules
```

负责功能

- 1. 发布任务入口；
- 2. 发布外卖分发任务；
- 3. 发布快递代取任务；
- 4. 发布私人任务；
- 5. 创建任务主单；
- 6. 创建任务明细；
- 7. 校验服务类型和节点是否匹配；
- 8. 我的任务列表；
- 9. 任务详情；
- 10. 管理员查看全部任务；
- 11. 任务取消前置判断。

关键业务逻辑

- 1. 任何任务都必须先写入 `tasks` 主表；
- 2. 不同任务类型写入不同明细表；
- 3. `service_type_id + node_id` 必须存在于 `service_node_rules` ；
- 4. 任务发布后初始状态建议为“已创建”；
- 5. 未支付任务可以取消；
- 6. 已接单任务取消需要走退款或管理员处理。

中文枚举要求

字段	可用值
tasks.urgent_flag	是、否
tasks.task_status	已创建、已支付、待接单、已接单、已取件、配送中、待签收、已完成、已取消、退款中

建议负责文件

```
Controllers/TaskController.cs
Services/TaskService.cs
Repositories/TaskRepository.cs
ViewModels/TaskCreateViewModel.cs
ViewModels/TaskListViewModel.cs
ViewModels/TaskDetailViewModel.cs
Views/Task/*
```

验收标准

- 1. 三类任务都能发布；
- 2. 主表和明细表都能正确插入；
- 3. 节点规则不匹配时禁止发布；
- 4. 我的任务能按当前用户显示；
- 5. 任务详情能显示对应明细；
- 6. 数据库变化可验证。

组员6：接单派单与状态流转负责人

负责表

assign_records
task_status_logs
tasks
runners

负责功能

- 1. 任务大厅；
- 2. 跑腿员抢单；
- 3. 防止重复抢单；
- 4. 管理员派单；
- 5. 管理员重派；
- 6. 跑腿员更新任务状态；
- 7. 用户确认签收；
- 8. 状态日志记录；
- 9. 状态历史查看；
- 10. 跑腿员工作状态更新。

状态流转建议

已支付
↓
待接单
↓
已接单
↓
已取件
↓
配送中
↓
待签收
↓
已完成

中文枚举要求

字段	可用值
assign_records.operation_type	自主接单、管理员派单、重新派单
runners.work_status	空闲、忙碌、离线

建议负责文件

```
Controllers/AssignController.cs
Controllers/TaskStatusController.cs
Services/AssignService.cs
Services/TaskStatusService.cs
Repositories/AssignRepository.cs
Repositories/TaskStatusLogRepository.cs
ViewModels/AssignRecordViewModel.cs
ViewModels/TaskStatusLogViewModel.cs
Views/Assign/*
Views/TaskStatus/*
```

验收标准

- 1. 跑腿员可以看到可接任务；
- 2. 抢单成功后生成接单记录；
- 3. 同一任务不能被重复抢单；
- 4. 每次状态变化都有日志；
- 5. 用户确认签收后任务变为“已完成”；
- 6. 跑腿员工作状态变化正确。

组员7：支付退款模块负责人

负责表

```
payments
refunds
tasks
assign_records
```

负责功能

- 1. 模拟支付页面；
- 2. 创建支付记录；
- 3. 支付状态查询；
- 4. 支付成功后推动任务状态；
- 5. 退款申请；
- 6. 退款审核；

- 7. 退款驳回；
- 8. 支付状态和退款状态联动；
- 9. 管理员查看退款列表。

关键业务逻辑

- 1. 未支付任务才允许支付；
- 2. 支付成功后任务进入“已支付”或“待接单”；
- 3. 已取消、已退款任务不能重复支付；
- 4. 退款申请必须关联支付记录；
- 5. 退款审核通过后支付状态应变为“已退款”；
- 6. 退款审核要保留处理结果和原因。

中文枚举要求

字段	可用值
payments.pay_method	微信、支付宝、现金
payments.pay_status	未支付、已支付、支付失败、已退款
refunds.refund_status	待审核、已通过、已拒绝

建议负责文件

```
Controllers/PaymentController.cs
Controllers/RefundController.cs
Services/PaymentService.cs
Services/RefundService.cs
Repositories/PaymentRepository.cs
Repositories/RefundRepository.cs
ViewModels/PaymentViewModel.cs
ViewModels/RefundViewModel.cs
Views/Payment/*
Views/Refund/*
```

验收标准

- 1. 用户可以模拟支付；
 - 2. 支付后 payments 表新增记录；
 - 3. 支付后任务状态正确变化；
 - 4. 用户可以申请退款；
 - 5. 管理员可以审核退款；
 - 6. 退款通过后支付状态正确变化。
-

组员8：评价投诉与信誉模块负责人

负责表

reviews
complaints
assign_records
runners

负责功能

- 1. 用户评价页面；
- 2. 提交评分和评价内容；
- 3. 评价列表；
- 4. 用户投诉页面；
- 5. 提交投诉；
- 6. 管理员投诉处理；
- 7. 投诉成立后扣减跑腿员信誉分；
- 8. 跑腿员评价统计；
- 9. 投诉状态查询。

关键业务逻辑

- 1. 只有已完成任务才能评价；
- 2. 一个接单记录最多一条评价；
- 3. 投诉必须关联接单记录；
- 4. 投诉处理结果应保存；
- 5. 投诉成立时影响跑腿员信誉分；
- 6. 信誉分不能随意低于系统允许范围。

中文枚举要求

字段	可用值
complaints.complaint_status	待处理、已成立、已驳回

建议负责文件

```
Controllers/ReviewController.cs
Controllers/ComplaintController.cs
Services/ReviewService.cs
Services/ComplaintService.cs
Repositories/ReviewRepository.cs
Repositories/ComplaintRepository.cs
ViewModels/ReviewViewModel.cs
ViewModels/ComplaintViewModel.cs
Views/Review/*
Views/Complaint/*
```

验收标准

- 1. 用户可以评价已完成任务；
- 2. 重复评价被阻止；
- 3. 用户可以提交投诉；
- 4. 管理员可以处理投诉；
- 5. 投诉成立后跑腿员信誉分变化；
- 6. 数据库记录完整。

组员9： 结算审计报表模块负责人

负责表

```
settlements
settlement_payment_items
audit_logs
audit_status_log_checks
audit_payment_checks
audit_refund_checks
reports
report_audit_items
```

负责功能

- 1. 跑腿员结算记录生成；
- 2. 结算明细查看；
- 3. 防止重复结算；
- 4. 审计日志查看；
- 5. 状态日志审计；
- 6. 支付记录审计；
- 7. 退款记录审计；
- 8. 订单数量统计；
- 9. 节点业务量统计；

10. 跑腿员绩效统计;
11. 退款率和投诉率统计;
12. 报表生成记录。

关键业务逻辑

1. 已完成且已支付的任务才进入结算;
2. 同一支付记录不能重复结算;
3. 有未处理投诉的任务暂不结算;
4. 报表可以先使用表格展示, 不强制做复杂图表;
5. 审计功能优先做查询和异常标记, 不必过度复杂。

建议负责文件

```
Controllers/SettlementController.cs
Controllers/AuditController.cs
Controllers/ReportController.cs
Services/SettlementService.cs
Services/AuditService.cs
Services/ReportService.cs
Repositories/SettlementRepository.cs
Repositories/AuditRepository.cs
Repositories/ReportRepository.cs
ViewModels/SettlementViewModel.cs
ViewModels/AuditViewModel.cs
ViewModels/ReportViewModel.cs
Views/Settlement/*
Views/Audit/*
Views/Report/*
```

验收标准

1. 可以生成跑腿员结算记录;
2. 可以查看结算明细;
3. 重复结算被阻止;
4. 可以查看审计日志;
5. 可以查看统计报表;
6. 管理员端能展示核心统计结果。

组员10: 测试文档与答辩负责人

负责范围

组员10负责测试、截图、文档和答辩材料, 不是简单“写文档”, 而是保证项目最后能讲得清、演示得顺。

主要任务

- 1. 制定测试用例；
- 2. 维护 Bug 记录表；
- 3. 协助每个模块做功能验收；
- 4. 用 SQL Developer 验证数据库变化；
- 5. 整理功能截图；
- 6. 整理演示流程；
- 7. 编写系统设计文档；
- 8. 汇总每个人的 PPT；
- 9. 准备答辩讲稿；
- 10. 组织演示彩排。

建议负责文件

```
Docs/test_cases.md
Docs/bug_list.md
Docs/demo_script.md
Docs/system_design.md
Docs/ppt_outline.md
Screenshots/*
```

验收标准

- 1. 每个模块至少有一组测试用例；
- 2. 每个核心流程至少有一张截图；
- 3. 每个 Bug 有记录和处理状态；
- 4. 答辩 PPT 内容完整；
- 5. 最终演示脚本能按顺序跑通。

七、模块协作关系

功能流程	主要负责人	配合人
项目创建与运行	组员1	全体
数据库建表与测试数据	组员2	全体
登录注册	组员3	组员1、组员2
地址管理	组员3	组员5
节点与服务类型	组员4	组员2
服务节点规则	组员4	组员5
跑腿员申请审核	组员4	组员6

功能流程	主要负责人	配合人
任务发布	组员5	组员3、组员4
模拟支付	组员7	组员5
任务大厅	组员6	组员5、组员7
抢单与派单	组员6	组员4
状态流转	组员6	组员5、组员7
退款处理	组员7	组员6
评价投诉	组员8	组员6、组员7
结算	组员9	组员6、组员7、组员8
审计报表	组员9	组员2
测试验收	组员10	全体
最终联调	组员1、组员10	全体

八、统一开发规范

1. 分层规范

后端统一采用：

```
Controller -> Service -> Repository -> OracleDbHelper -> Oracle
```

说明：

层	职责
Controller	接收请求、返回页面、处理跳转
Service	编写业务规则、状态判断、权限判断
Repository	编写 SQL、访问数据库
ViewModel	页面表单和展示数据
View	页面展示，不直接访问数据库

2. 命名规范

建议每个模块保持一致命名：

```
XxxController.cs
XxxService.cs
XxxRepository.cs
XxxViewModel.cs
Views/Xxx/*.cshtml
```

例如任务模块：

```
TaskController.cs
TaskService.cs
TaskRepository.cs
TaskCreateViewModel.cs
Views/Task/Create.cshtml
Views/Task/Details.cshtml
```

3. 禁止事项

1. 不允许每个人重新创建一个 MVC 项目；
2. 不允许绕过 Service 直接在 Controller 中写复杂 SQL；
3. 不允许在 View 中访问数据库；
4. 不允许随意修改其他人的模块；
5. 不允许修改数据库中文枚举为英文值；
6. 不允许随意改动表结构；
7. 不允许直接提交无法编译的代码；
8. 不允许把 Oracle 原始异常直接显示给用户；
9. 不允许使用 `update`、`fix`、`111` 这类无意义提交信息；
10. 不允许最后一天才开始联调。

4. 中文枚举总表

类型	中文值
用户角色	普通用户、跑腿员、管理员
账号状态	正常、禁用
是否标志	是、否
节点状态	正常、关闭
服务类型状态	启用、禁用
跑腿员审核状态	待审核、已通过、已拒绝
跑腿员工作状态	空闲、忙碌、离线

类型	中文值
任务状态	已创建、已支付、待接单、已接单、已取件、配送中、待签收、已完成、已取消、退款中
接派类型	自主接单、管理员派单、重新派单
支付方式	微信、支付宝、现金
支付状态	未支付、已支付、支付失败、已退款
退款状态	待审核、已通过、已拒绝
投诉状态	待处理、已成立、已驳回

禁止写成：

USER
RUNNER
ADMIN
NORMAL
DISABLED
CREATED
PAID
WAITING
Y
N

九、Git 分支建议

分支	负责人	用途
main	组员1	稳定可演示版本
dev	组员1	日常合并版本
feature/base-db	组员2	数据库脚本和测试数据
feature/account-address	组员3	账户、权限、地址
feature/base-runner	组员4	基础资料、跑腿员
feature/task	组员5	任务发布
feature/assign-status	组员6	接单派单、状态流转
feature/payment-refund	组员7	支付退款
feature/review-complaint	组员8	评价投诉
feature/settlement-report	组员9	结算审计报表
feature/test-docs	组员10	测试文档答辩

提交信息建议：

完成用户登录功能
完成地址管理页面
完成任务发布主单插入
修复抢单重复判断
补充支付测试数据
整理答辩演示截图

十、每个模块的完成标准

每个成员完成模块时，至少满足：

- 1. 页面能打开；
- 2. 表单能提交；
- 3. 数据能写入或查询；
- 4. 数据库对应表变化正确；
- 5. SQL Developer 可以验证；
- 6. 角色权限正确；
- 7. 中文枚举正确；
- 8. 错误提示友好；
- 9. dotnet build 通过；
- 10. 至少提供一张功能截图；
- 11. 能讲清自己负责的表和业务逻辑。

十一、优先级建议

如果时间充足，按完整方案做。
如果时间紧，按以下优先级裁剪：

优先级	必做内容
P0	项目能运行、数据库能连、登录能进系统
P0	普通用户发布任务
P0	支付后任务可被接单
P0	跑腿员抢单和状态更新
P0	用户确认签收
P1	评价、投诉、退款
P1	管理员审核跑腿员
P1	节点、服务类型、规则管理
P2	结算、审计、报表
P2	页面美化、统计图、复杂筛选

最低可答辩版本必须跑通：

登录 -> 发布任务 -> 支付 -> 接单 -> 更新状态 -> 签收 -> 查看数据库变化

十二、答辩材料分配

答辩内容	负责人
项目背景与需求分析	组员10 统筹，全体补充
系统整体架构	组员1
数据库设计	组员2
登录权限与地址管理	组员3
基础资料与跑腿员审核	组员4
任务发布流程	组员5
接单派单与状态流转	组员6
支付退款流程	组员7
评价投诉与信誉分	组员8
结算审计报表	组员9
测试过程与演示脚本	组员10
最终系统演示	组员1、组员10 统筹，全体配合

十三、总结

新版分工按照“项目完全未开始”的情况重新设计，重点从原来的“已有基础上继续补模块”改为：

- 1. 先搭项目骨架；
- 2. 先建数据库和测试数据；
- 3. 再做登录权限和基础资料；
- 4. 然后并行开发任务、接单、支付、评价投诉、结算报表；
- 5. 最后集中联调、测试、截图和答辩。

这种分工更适合从零启动，因为每个人都有明确模块、明确表、明确页面、明确验收标准，也能避免最后所有人都等总集成负责人补基础代码。